VeriMM: A Large-Scale Multimodal Verilog Dataset with High-quality Testbenches for Hardware Code Generation

Siyu Yun, Qingsong Peng, Yu Li†, Cheng Zhuo
Zhejiang University
Hangzhou, China

Abstract

Large language models for hardware design are fundamentally limited by the quality of training data. Existing automated datasets suffer from a critical verification failure: 95.1% of generated testbenches in the largest corpus unconditionally report "pass," rendering them ineffective for functional validation. Meanwhile, real-world RTL development relies on multimodal specifications (e.g., timing diagrams, schematics, FSMs), which are largely absent from current datasets. The lack of reliable verification signals and multimodal grounding hinders progress in LLM-based hardware code generation.

We present VeriMM, a large-scale multimodal Verilog dataset with verified testbenches. VeriMM contains 57K hardware designs, each paired with textual and visual specifications, RTL implementations, and executable testbenches. We propose a fully automated construction pipeline featuring (1) a template-based testbench generation mechanism that eliminates structural failures by design, and (2) a coverage-driven feedback loop that improves stimulus diversity and verification strength. This pipeline achieves state-of-the-art bug-detection performance (F1 = 0.8253) while reducing token cost by 99% compared to the prior best method. We evaluate VeriMM by fine-tuning three state-of-the-art multimodal LLMs for hardware code generation. Models trained on VeriMM's verification-enforced data achieve up to 21.3% improvement in functional pass rate over equally sized datasets without verification, furthermore, the inclusion of visual modalities yields a average gain of 15.4%. These results show the effectiveness of verification correctness and multimodal grounding. VeriMM and the full pipeline are available at: https://github.com/SeRAlab/VeriMM-Dataset.

Keywords

Dataset, Multimodal, Testbench Generation, Specification

1 Introduction

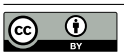
While Large Language Models (LLMs) have demonstrated significant proficiency in software generation [5, 28], they still struggle to produce accurate hardware code. This limitation stems from the scarcity, inconsistency, and weak verification of open-source hardware repositories. Consequently, large-scale and high-fidelity hardware datasets are essential for improving LLM performance in neural hardware code generation tasks.

The typical process of Verilog design begins with interpreting circuit specifications, which describe functional requirements such as input/output port definitions and expected outputs under specific input sequences [13]. In real-world workflows, these specifications are inherently multimodal, integrating textual descriptions with circuit schematics, timing diagrams, and finite-state-machine (FSM) representations. When using uni-modal LLMs for Verilog generation, engineers must manually convert visual information into textual form to avoid information loss, resulting in significant overhead and potential semantic distortion [12]. In contrast, multimodal models can directly reason over heterogeneous specification formats, enabling unified interpretation of textual and visual signals [8]. This shift highlights an increasing demand for multimodal training datasets that faithfully reflect real-world hardware specifications.

Despite recent progress, existing datasets for neural hardware code generation remain largely unimodal and weakly grounded in executable verification. Multimodal Verilog datasets are extremely scarce—for instance, [2] contains only 30 circuits, each with only one type of visual modality (either a schematic or an FSM diagram)—severely limiting MLLMs' ability to interpret complex specifications and generalize across hardware designs. Meanwhile, most prior works construct training data from LLM-generated specifications or RTL code [4, 15]. Although some methods apply functional filtering [7, 36], the resulting datasets often exhibit weak alignment between specifications and implementations. In the absence of explicit verification artifacts such as high-quality testbenches, models trained on these datasets tend to capture superficial correlations rather than functionally grounded mappings between specifications and code.

To address these limitations, we introduce VeriMM, a large-scale multimodal dataset for neural hardware code generation that integrates textual specifications, RTL implementations, visual representations, and high-quality testbenches. Each data instance in VeriMM consists of text, code, and visual modalities. We construct VeriMM through an automated pipeline comprising three stages: data preprocessing, testbench generation, and image generation. Starting from aggregated open-source repositories, we perform syntax validation and deduplication to prepare the initial candidates. We then generate structurally sound testbenches to verify each design; only those that pass this rigorous functional verification are retained, resulting in a final collection of 57K high-quality, verified specification–code pairs. For these validated designs, we further synthesize three types of visual representations using a combination of open-source and commercial tools. By integrating executable testbenches, VeriMM provides a functionally grounded framework

†Corresponding author: Yu Li. Email: li.yu@zju.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.
ICCAD '26, San Jose, CA, USA
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2873-0/2026/11
https://doi.org/10.1145/3831252.3834170

Table 1: Comparison of Verilog datasets. VeriMM is the only dataset that is simultaneously large-scale, fully multimodal, and equipped with high-quality automatically generated testbenches.

Dataset	# Circuits	DUT	Specification Modalities				Testbench	Automated Generation
			Text	Timing	Schematic	FSM		
Large-scale • No testbench								
PyraNet [15]	692k	✔	✔	×	×	×	–	✔
Origen [4]	222k	✔	✔	×	×	×	–	✔
FreeSet [1]	175k	✔	×	×	×	×	–	✔
Automated testbench • Low quality								
AutoVCoder [7]	23k	✔	✔	×	×	×	**Low** (95.1% blind-pass)	✔
VeriReason [31]	1.85k	✔	✔	×	×	×	**Low** (bug detection rate)	✔
High-quality testbench • Manual • Small scale								
CVDP [17]	176	✔	✔	×	×	×	High Quality	×
VerilogEval [12]	156	✔	✔	×	×	×	High Quality	×
RTLLM [14]	50	✔	✔	×	×	×	High Quality	×
ChipGPT-V [2]	30	✔	✔	×	✔	✔	High Quality	×
VeriMM (this work) — large-scale • fully multimodal • automatically verified								
VeriMM (Ours)	**57k**	✔	✔	✔	✔	✔	**High Quality**	✔

that aligns hardware specifications, implementations, and visual modalities. Table 1 shows the comparison between VeriMM and other open source datasets.

Our contributions are summarized as follows:

- We introduce VeriMM, a large-scale multimodal dataset for neural hardware code generation, integrating textual specifications, Verilog implementations, visual representations, and executable testbenches across 57K designs.
- We propose an automated testbench generation pipeline that aligns specifications with RTL implementations (e.g., Verilog) using template-based execution and coverage-driven refinement, producing high-quality executable testbenches for reliable functional verification.
- We develop a scalable pipeline that generates three visual modalities (timing diagrams, schematics, and FSM diagrams) using a combination of open-source and commercial tools, producing specification-aligned representations that bridge RTL code and hardware documentation.

To validate the dataset's utility, we fine-tune three state-of-the-art MLLMs using SFT. Our experimental results on the test set demonstrate that incorporating these multimodal elements significantly enhances the models' ability to interpret and design complex hardware. We shall open-source our dataset and code upon acceptance.

2 Related Works

2.1 Testbench Generation

AutoBench [19] employs an LLM to generate a Python-based reference model from the circuit specification, followed by the generation of specific test scenarios and their corresponding input stimuli. This verification flow records the Design Under Test (DUT) output for each scenario and compares it against the reference model's output; a discrepancy between the two indicates a bug. However, the primary limitation of this approach is its dependency on the reference model's accuracy: if the LLM generates an erroneous Python model, the resulting testbench will produce incorrect verification results.

CorrectBench [20] addresses the issue of buggy reference modules by introducing a validation stage into the AutoBench pipeline. This approach utilizes an LLM to generate multiple candidate DUTs and evaluates their performance across various scenarios. By implementing specific criteria to reject faulty reference models and iteratively regenerate them based on simulation feedback, CorrectBench significantly enhances testbench accuracy.

The main problem of CorrectBench is its binary "all-or-nothing" approach to suspicious testbenches, which often forces a choice between discarding valuable scenarios or accepting faulty ones. ConfiBench [18] solves this problem by shifting from coarse testbench-level validation to fine-grained, confidence-based scenario masking. By utilizing an RTL-Scenario matrix to quantitatively evaluate and mask individual low-credibility scenarios, ConfiBench preserves the reliable portions of multiple testbenches. These validated fragments are then integrated through a multi-testbench ensemble, maximizing functional correctness without the information loss inherent in CorrectBench's original filtering process.

As a SOTA algorithm for testbench generation, ConfiBench [18] incurs significant token expenditures and substantial latency per testbench. To facilitate testbench generation for large-scale Verilog datasets, a more efficient and accurate workflow is required. We address this challenge by introducing a generation flow utilizing pre-designed testbench templates and coverage-gap reporting. Consequently, our proposed flow is exceptionally efficient, requiring only 1% of the tokens consumed by ConfiBench while maintaining superior accuracy.

2.2 Verilog Dataset Generation

Testbench in dataset. Existing Verilog datasets have evolved from simple RTL collections [1, 25] to code-specification pairs intended for SFT [4, 15]. However, the generation of functional testbenches

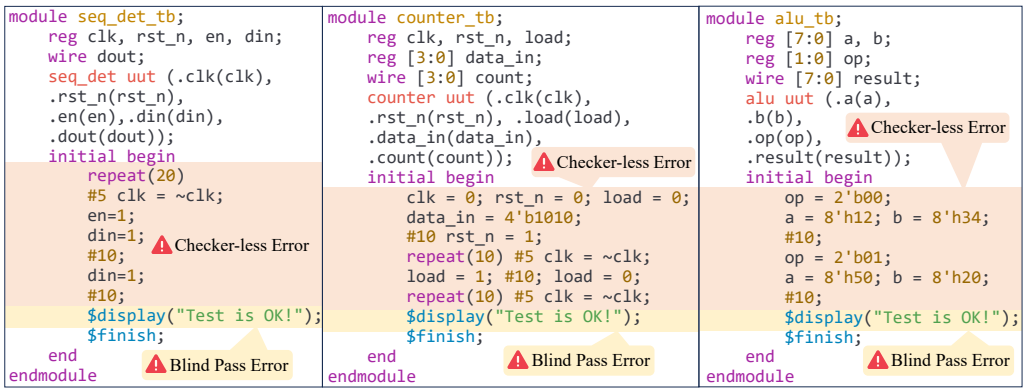


Figure 1: Representative failure modes of LLM-generated testbenches. Each case suffers from two critical flaws: Checker-less Error, where the code provides stimuli but does not verify the output value of DUT, and Blind Pass Error, which always reports success.

remains a bottleneck. While datasets like AutoVCoder [7] attempt to simultaneously generate code, specifications, and testbenches from keywords, the resulting testbenches often suffer from fatal structural flaws. We identify two primary failure modes in LLM-generated testbenches: "Blind Pass", where the environment reports success regardless of DUT behavior, and "Checker-less", characterized by the total absence of output validation logic (e.g., ==, !=). Fig. 1 shows three examples with these two failure modes. We analyze all the testbench from AutoVCoder and find 95.1% of the testbenches have "Blind Pass" error and 45.0% have "Checker-less" error. VeriReason [31] generates testbenches directly from Verilog design modules for use in Reinforcement Learning (RL). This approach deviates from standard verification flows where testbenches are derived from specifications; consequently, these testbenches may mirror implementation bugs, leading to inaccurate feedback during training. These structural and logical deficiencies compromise the integrity of current large-scale datasets, as the lack of rigorous verification leads to LLMs that fail to generalize across diverse and complex hardware designs.

Multimodal dataset. A typical specification for a circuit is a file with multimodal content, namely text and image. All the aforementioned works in this section only have text modality. This study [2] propose a multimodal Verilog benchmark dataset for evaluating the ability of an MLLM in Verilog design task. The dataset contains only 30 circuits, each circuit sample has four components: code, specification, testbench, a PNG image (FSM or schematic). Since both the testbench and images in this dataset are manually created, scaling it to a larger size presents a significant challenge. In addition, the timing diagram, which shows the port behavior across multiple clock cycles are absent in this dataset.

In contrast, our dataset is automatically generated, providing high-quality testbenches and rich multimodal annotations (timing diagrams, schematics, and FSM) at large scale.

3 Method

As illustrated in Fig. 2, our multimodal dataset generation flow consists of three primary stages: data preprocessing, testbench generation, and image generation. During preprocessing, we aggregate open-source specification-code pairs and perform rigorous syntax validation to exclude erroneous designs. We then deduplicate the remaining modules and append metadata tags to the verified pairs.

Following preprocessing, we validate the functional consistency between the specifications and Verilog code. We first leverage the reasoning capabilities of an LLM to generate initial testcases based on the textual specifications. After simulating these cases against the DUT, we extract a coverage gap report from the simulator. This report is then fed back to the LLM to iteratively generate targeted testcases that address the identified gaps. Finally, all testcases are integrated into a comprehensive testbench for thorough verification.

For sequential circuits, we utilize a passing testcase to generate and render a timing diagram of all I/O ports into a PNG image. To produce schematics, we employ an open-source synthesis tool to generate a gate-level netlist, which is then rendered as a graphical representation. For designs featuring FSMs, we utilize a commercial tool to extract the FSM logic and render it into a standardized image format that clearly labels state names and transitions.

3.1 Data Preprocessing

Numerous studies have open-sourced uni-modal Verilog datasets [4, 7, 31], providing the research community with ready-to-use code-specification pairs. While data generation flows and sample components vary across these works, most provide a core foundation of textual specifications and corresponding RTL code. Consequently, our generation pipeline begins by aggregating and unifying these code-specification pairs.

After collecting the initial code-specification pairs, we utilize Icarus Verilog [32] to perform syntax validation, discarding any erroneous entries. To ensure dataset diversity, we address the redundancy inherent in aggregating from sources like GitHub and

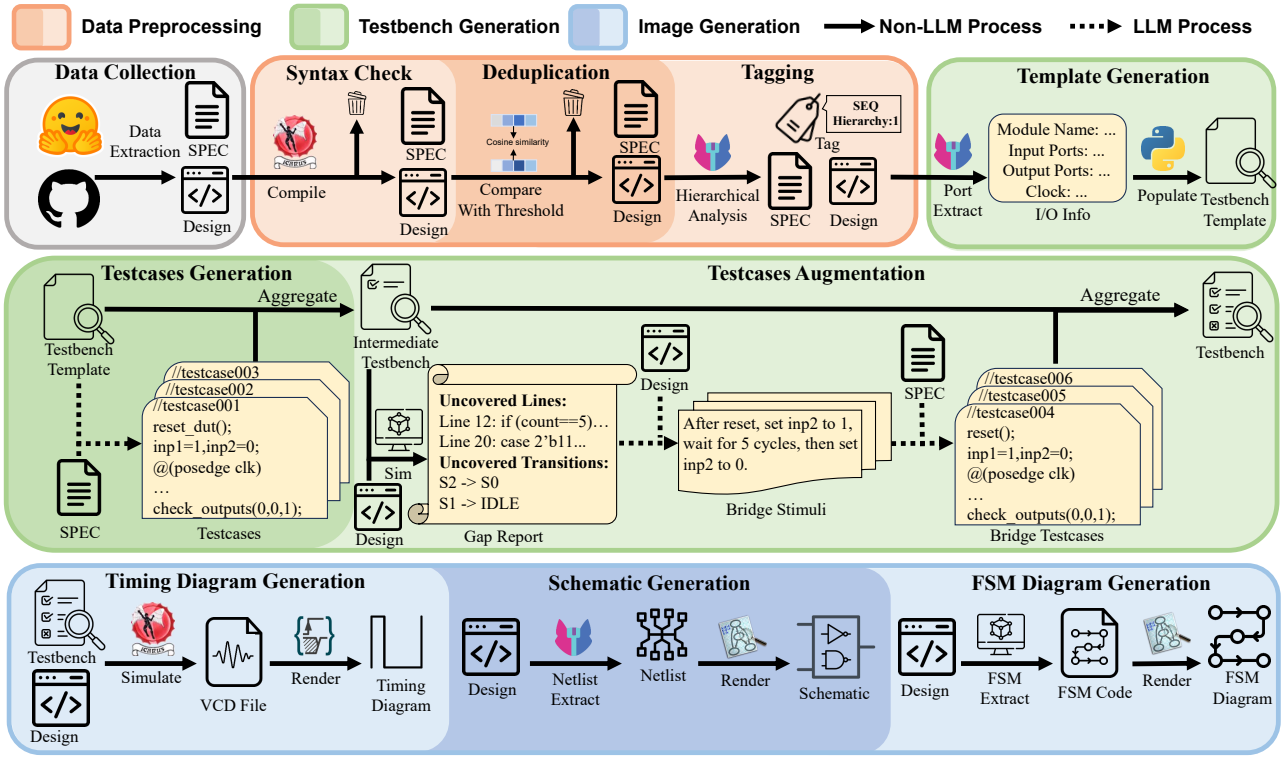


Figure 2: Overview of our multimodal dataset construction pipeline *VeriMM*, integrating specification–code pairs, executable testbenches, and visual modalities. In the testbench generation pipeline, an initial set of three testcases is first generated and integrated with a template to form an intermediate testbench. Based on the gap report generated after simulation, three additional testcases are generated to cover the identified gaps. Finally, all six testcases are integrated into the final comprehensive testbench.

OpenCores [16]. We employ a sentence transformer to generate embeddings for each module, removing duplicates where the cosine similarity exceeds a predefined threshold. This embedding-based approach effectively filters modules that are functionally identical but differ in minor formatting details, such as indentation or whitespace.

To facilitate further processing, we append two metadata tags to each circuit: circuit type (sequential or combinational) and hierarchy. The circuit type is determined by detecting edge-sensitive event controls, such as `@(posedge)` or `@(negedge)`. This tag guides the subsequent testbench generation process. The hierarchy tag, extracted via Yosys [33], defines the level of detail and guides the schematic generation process.

3.2 Testbench Generation

The automation of hardware verification via LLMs faces three fundamental challenges: ensuring a robust simulation infrastructure that avoids deceptive success reports, maintaining the integrity of the verification oracle against buggy design leakage, and achieving comprehensive scenario coverage without human intervention. Existing automated approaches often struggle with "Blind Pass" errors, where a testbench erroneously reports success due to a lack of rigorous checking logic, or suffer from contamination, where the model inadvertently adopts flawed design behavior as the ground truth. Our core insight lies in decoupling the execution framework from the functional logic. By first establishing a deterministic template and then isolating the model from the design implementation during reference value inference, we ensure that the verification process remains an independent and rigorous arbiter of correctness. Furthermore, we leverage a closed-loop feedback mechanism that converts structural coverage gaps into natural language stimuli descriptions, enabling the LLM to systematically explore unverified corner cases while preserving a clean separation between DUT analysis and reference value inference.

The proposed testbench generation algorithm follows a structured three-stage workflow to ensure functional verification with high coverage and reliability, as illustrated in Fig. 2. The process begins with **Template Generation**, a deterministic stage that operates independently of LLMs. Utilizing Yosys [33] for precise metadata extraction, we identify module name, I/O port names and clock polarities from the Verilog design. This metadata populates a standardized testbench template that provides a robust execution infrastructure, as shown in Fig. 3. To eliminate "Checker-less" and

```
1 module <top_name>_tb;
2     /* 1. Port Declarations & Verification Tracking */
3     reg clk, rst_n, <inputs>;
4     wire <outputs>;
5     integer pass_cnt, fail_cnt;
6
7
8     /* 2. DUT Instantiation */
9     <top_name> dut ( .clk(clk), .rst_n(rst_n), .* );
10
11
12    /* 3. CLock Generation */
13    initial begin clk = 0; forever #5 clk = ~clk; end
14
15
16    /* 4. Check_outputs Task Definition */
17    task check_outputs(input <expected_vals>);
18        if (<actual> == <expected>) pass_cnt++;
19        else fail_cnt++;
20    endtask
21
22
23    /*5. Empty Tasks Reserved For LLM To Fill In */
24    task reset_dut; endtask
25    task testcase001; endtask
26
27
28    /* 6. Structured Stimulus Execution */
29    initial begin
30        {pass_cnt, fail_cnt} = 0;
31        reset_dut();
32        testcase001();
33        report_summary(pass_cnt, fail_cnt);
34        $finish;
35    end
36
37    /* 7. Safety & Debug Infrastructure */
38    initial #TIMEOUT_LIMIT $finish; // Watchdog timer
39    initial begin $dumpfile("..."); $dumpvars(0, ...); end
40 endmodule
```

Figure 3: Simplified representation of our structured testbench template. Lines 24 and 25 are reserved for testcases, which are automatically generated by the LLM to verify specific DUT behaviors.

"Blind Pass" errors, the template enforces a strict verification protocol: it instantiates a specialized `check_outputs` task (line 17 to 20 in Fig. 3) that performs real-time discrepancies monitoring between DUT outputs and reference values, updating internal `pass_cnt` and `fail_cnt` registers. Furthermore, a watchdog timer (line 38 in Fig. 3) is integrated to preemptively terminate simulations in the event of infinite loops. This stage ensures that the resulting testbench is not merely a sequence of stimuli but a rigorous verification environment capable of reporting substantive success or failure.

Building upon this infrastructure, the second stage, **Testcases Generation**, focuses on generating functional stimuli while neutralizing the risk of "Buggy Code Leakage". In this stage, the LLM is provided only with the natural language specification and the structural testbench template, intentionally isolating it from the DUT implementation. By depriving the model of the potentially flawed hardware code, we ensure that the derived expected output values are anchored solely in the "golden reference" of the specification. The LLM interprets the functional requirements to identify critical scenarios, converting them into specific Verilog tasks compatible with the template's placeholder interface. Each task is mandated to initialize the DUT via a `reset_dut` task and conclude by invoking `check_output` with independently inferred reference arguments. This isolation strategy ensures that hardware bugs do not inadvertently corrupt the verification oracle, maintaining a clean separation between the design under test and the ground truth. The LLM generated testcases are aggregated to the testbench template and make the template a "Intermediate Testbench" which is ready to simulate.

The final stage, **Testcases Augmentation**, establishes a feedback loop to address the "Which scenarios to check" challenge and maximize stimulus coverage. The "Intermediate Testbench" is executed on a simulator, and a script is used to distill the simulation coverage metrics (e.g., branch coverage, line coverage, and toggle coverage) into a human-readable "Gap Report", highlighting unexecuted lines and unvisited FSM transitions. During this augmentation phase, the LLM is permitted to analyze the DUT implementation alongside the "Gap Report" to understand the root causes of coverage deficiencies. To maintain verification integrity and avoid leakage, we restrict the model's output in this context to "natural language stimuli" descriptions rather than direct Verilog code with inferred outputs, those "natural language stimuli" are called "Bridge Stimuli" in Fig. 2. Subsequently, a separate LLM process, operating under the aforementioned isolation logic, converts these natural language stimuli into executable "Bridge Testcases." The specification is used in this process, and the design's expected outputs must be inferred. Finally, the template and all verified testcase tasks are aggregated into a single, comprehensive testbench. Upon final execution, the resulting report provides high-fidelity feedback regarding functional correctness, serving as a critical signal for iterative design refinement.

3.3 Image Generation

Timing diagram. Timing diagrams are commonly included in specifications to illustrate the timing relationships between various ports during specific functional operations. To automate the generation of timing diagrams for a multimodal dataset, we implemented a Python-based workflow that integrates industry-standard simulation and visualization tools. For a sequential circuit, this process begins by compiling the DUT and its corresponding testbench using Icarus Verilog [32], followed by simulation to generate a Value Change Dump (VCD) file. Then we use the vcd2wavedrom library [26] to parse the VCD data and convert signal transitions into a structured JSON format. Finally, the WaveDrom engine[3] renders this data into a PNG image. Fig. 4 (left) illustrates two examples of these generated waveforms.

Schematic. Schematics provide a direct structural overview of a circuit and are prevalent in specifications, particularly for hierarchical designs. In multi-module circuits, sub-modules are typically represented as blocks interconnected by orthogonal signal lines with annotated port names. For lower-complexity designs, these schematics often detail the individual logic gates and their specific interconnections.

To generate schematic corresponding to typical specification, and consider all the aforementioned rules, we propose the following process. The process begins by utilizing Yosys [33] to synthesize the Verilog code into a JSON-formatted netlist. This netlist contains different kinds of logic units (e.g., AND gates, OR gates, D flip-flops, and multiplexers). We categorize a circuit with fewer than eight

gates as a simple circuit and render it to an SVG file using netlistsvg [27], then convert it to a PNG file with Inkscape [11].

For a circuit with more than eight gates in the JSON-formatted netlist, or a circuit with more than one hierarchy level, we first invoke Yosys [33] to export the module structure and port connectivity relationships. This metadata is mapped into the Graphviz DOT language [6], which defines a directed graph where each module is represented as a central rectangular block connected to its respective input and output signals. Finally, the Graphviz library renders this representation in PNG format, employing orthogonal routing of signals and a left-to-right layout to ensure professional visual clarity. Two examples of generated schematic are shown in Fig. 4 (middle).

FSM diagram. FSMs constitute a universal design methodology for diverse sequential circuit applications. FSM diagrams are frequently included in specifications to clarify complex logic; therefore, incorporating these diagrams into a multimodal dataset significantly enhances an MLLM's ability to interpret and design sophisticated hardware.

We first use Verdi [22] to identify state registers and transition conditions, exporting this behavioral logic into a structured intermediate format denoted as "FSM Code." A custom Python-based parser then translates this information into the DOT graph description language. Finally, Graphviz is employed to render high-fidelity, vector-based state diagrams in PNG format. This integrated flow ensures that the visualized FSM accurately reflects the hardware implementation while providing a clear, human-readable representation of complex control logic. Two examples of generated FSM diagrams are shown in Fig. 4 (right).

4 Experiment

Our multimodal dataset generation flow consists of three primary stages: data preprocessing, testbench generation, and image generation. To evaluate this pipeline, we first validate our testbench generation flow by conducting comparative experiments on established public datasets to demonstrate its superior performance over previous methods. We further construct a large-scale multimodal dataset and fine-tune three MLLMs on our dataset; the resulting performance improvements underscore the dataset's practicality and effectiveness in real-world circuit design tasks.

4.1 Effectiveness of Testbench Generation

We evaluate our testbench generation algorithm against a direct generation baseline—where an LLM generates a testbench from a specification in a single step—and four prominent previous works: VeriReason [31], AutoBench [19], CorrectBench [20], and the current state-of-the-art, ConfiBench [18]. While VeriReason derives testbenches from existing Verilog code, we include it to highlight the superior quality of our specification-driven dataset. AutoBench and CorrectBench are earlier versions of ConfiBench.

In addition to the six methods previously mentioned, we include two ablation studies: the first removes the testcases augmentation step to evaluate performance without coverage report instructions, while the second excludes testbench template generation to isolate the effectiveness of the template itself.

Table 2: Performance and cost-efficiency comparison of testbench generation algorithms.

Algorithm	Accuracy	Precision	Recall	F1 Score	Token Cost
Direct Generation	0.8675	0.7662	0.4117	0.5359	1079
VeriReason [31]	0.8696	0.7017	0.5111	0.5917	**912**
AutoBench [7]	0.8695	0.6725	0.5948	0.631	14610
CorrectBench [20]	0.9235	0.7751	0.8237	0.7989	37538
ConfiBench [18]	0.9309	0.7970	0.8381	0.8169	393841
Ours w/o Aug	0.9039	0.7711	0.6803	0.7231	1357
Ours w/o Tmp	0.8742	0.7059	0.5796	0.6369	3205
Ours	**0.9336**	**0.7989**	**0.8543**	**0.8253**	4318

"Aug" denotes testcases augmentation.
"Tmp" denotes testbench template.

To evaluate our method, we constructed a benchmark of 1,960 circuits by augmenting the VerilogEval [12] and RTLLM [14] datasets with mutant code from AutoBench [19]. Each sample was annotated with correct/incorrect labels against human-written "golden" testbenches. Experiments were conducted using Gemini 3 Flash Preview [9] and Qwen-2.5-Coder-32B [10, 34], representing high-performance general-purpose and specialized code-centric LLMs, respectively. We report the average results of three independent runs, with the performance of Gemini 3 Flash detailed in Table 2. Details for Qwen-2.5-Coder are omitted for brevity, though its confidence intervals and F1 scores are illustrated in Fig. 5.

Analysis of the experimental results demonstrates that our proposed method consistently achieves state-of-the-art performance across both language models. As shown in Table 2, the "Ours" configuration reaches a peak F1 score of 0.8253 on the Gemini model, significantly surpassing the 0.5359 achieved by the "Direct Generation" baseline. The ablation studies, "Ours w/o Aug" and "Ours w/o Tmp," further validate the design, showing that the integration of structural templates and coverage-guided augmentation is essential for maximizing bug-detection capabilities. Fig. 5 shows the F1 score improvements over the most competitive baselines are numerically stable. The "Ours" framework provides a more balanced performance across accuracy and recall, ensuring that fewer bugs go undetected during the verification process.

The trade-off between token cost and F1 score is illustrated in Fig. 6. A significant advantage of our framework is its exceptional economic efficiency. While competitive baselines like ConfiBench incur an exorbitant cost of 393,841 tokens per benchmark, our algorithm achieves a superior F1 score of 0.8253 with only 4,318 tokens—a nearly 100-fold reduction in resource expenditure. This efficiency stems from using deterministic scripts for structural extraction, allowing the LLM to focus solely on critical stimulus gaps. Consequently, our approach is uniquely positioned for the cost-effective cleaning of massive datasets containing millions of samples, providing a scalable foundation for hardware-related LLM training.

4.2 Multimodal Dataset Quality Analysis

We aggregated data from four highly cited sources: AutoVCoder [7], VeriReason [31], LLM_EDA [29], and Haven [35], totaling 89k Verilog code-specification pairs. We use the paraphrase-MiniLM-L3-v2 sentence transformer [21] with a similarity threshold of 0.95 for deduplication. We use Gemini 3 Flash as the LLM to generate

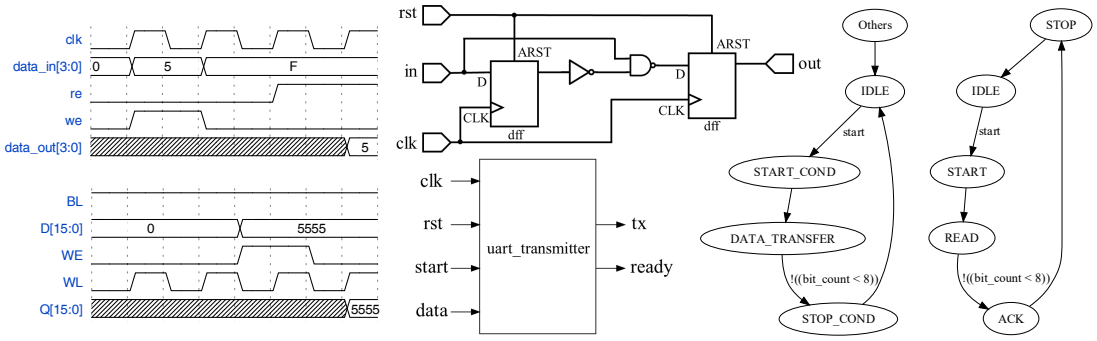


Figure 4: Timing diagram, schematic and FSM diagram examples generated by our method.

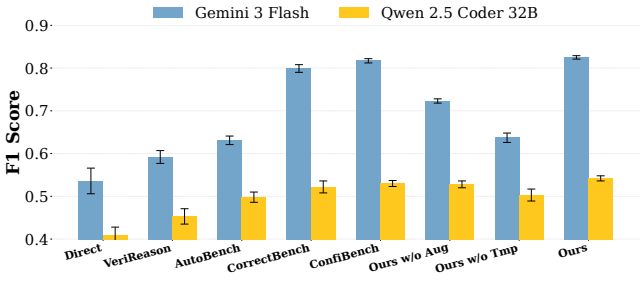


Figure 5: F1 score comparison with 95% confidence intervals across different LLMs.

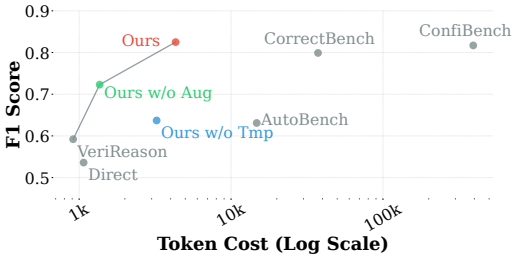


Figure 6: Performance vs. economic efficiency on Gemini 3 Flash.

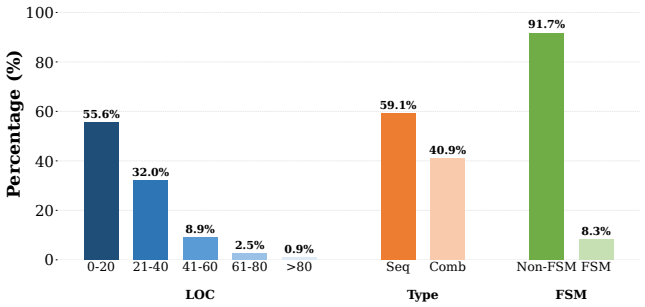


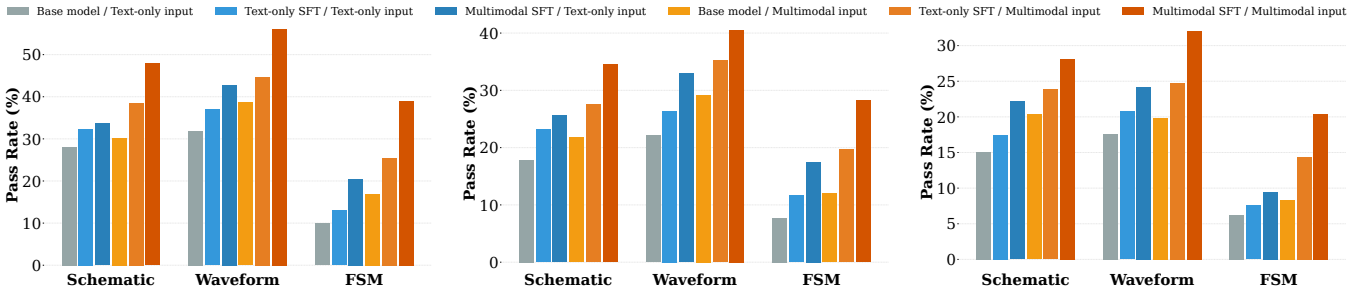
Figure 7: Statistical characteristics of our multimodal Verilog circuit dataset, including LoC distribution, the proportion of combinational/sequential circuits, and the percentage of circuits containing FSM.

testbench. After verification with testbench and discard the buggy designs, we achieve a clean set of 57k pairs, the characteristics of the final dataset are detailed in Fig. 7.

To evaluate the effectiveness of our multimodal dataset, we conducted SFT on three open-source MLLMs, namely Qwen3-VL-8B [23], GLM-4.6V-Flash (9B) [24], and InternVL3.5-8B [30]. For each original MLLM and each type of image (timing diagram, schematic, and FSM diagram), we conducted two SFTs: one with an image as input and one without an image as input. The dataset was randomly divided into a training set and a test set at a ratio of roughly 9:1. To evaluate the Verilog generation capability of these models (both base models and those fine-tuned with our dataset), we used test-set specifications (once with an image and once without an image) to prompt the MLLMs, then used the testbenches in that set to simulate the generated circuit designs. We calculated the pass rates and present them in Fig. 8.

The experimental results demonstrate that SFT significantly enhances the performance of MLLMs in hardware design tasks across all evaluated categories, including schematic, waveform, and FSM. Across all three models—Qwen3-VL-8B, GLM-4.6V-Flash, and InternVL3.5-8B—the fine-tuned versions consistently outperform their respective base models when provided with identical input modalities. For instance, using Qwen3-VL-8B with text-only input, the fine-tuned model improves the FSM accuracy from 9.96% to 12.97%, while the image-based SFT further elevates the performance to 20.47%. This trend confirms that specialized fine-tuning with high-quality, verified hardware data effectively aligns general-purpose MLLMs with the rigorous logical and structural requirements of Verilog specifications.

Furthermore, the data highlights the critical role of visual information during the inference stage, as the inclusion of image inputs consistently yields superior outcomes compared to text-only prompts. For every model tested, providing both image and text



(a) Qwen3-VL-8B. (b) GLM-4.6V-Flash (9B). (c) InternVL3.5-8B.

Figure 8: Pass rate comparison across three MLLMs under different fine-tuning and input modalities.

inputs leads to a substantial performance leap over using text alone, regardless of the fine-tuning state. Consequently, the optimal configuration is observed when the models are fine-tuned on multimodal data and subsequently provided with both textual specifications and visual diagrams during testing. This "Multimodal SFT / Multimodal input" synergy achieves the highest accuracy across all benchmarks, such as reaching 56.12% for waveforms on Qwen3-VL-8B, thereby validating the necessity of a fully multimodal approach for complex hardware-related reasoning.

To further evaluate the impact of data quality on model performance, we conducted a comparative study using the Qwen3-VL-8B model, focusing on the differences between our verified dataset and raw, unfiltered alternatives. This experiment aims to determine whether the performance gains are driven by our high-quality testbench filtering process or simply by the volume of data. We established three experimental groups using an identical test set: the Full Dataset containing all collected raw samples, a Random Subsample group that reduces the raw data volume to match the size of our filtered set, and our Filtered (Ours) dataset.

The experimental results in Table 3 underscore that for hardware-related multimodal training, data quality significantly outweighs raw quantity. While simple random subsampling leads to a performance decline, our Filtered (Ours) dataset achieves substantial gains over the full raw dataset, with pass rate improvements of +16.47% for Schematic, +17.28% for Waveform, and +21.91% for FSM. These results validate that our testbench-driven cleaning process effectively eliminates "buggy" samples, providing a higher-fidelity foundation for MLLM fine-tuning than even the much larger unfiltered alternatives.

Table 3: Pass rate comparison between our filtered dataset and raw alternatives on Qwen3-VL-8B.

Dataset	Schematic	Waveform	FSM	Average
Full Dataset	31.42%	38.84%	17.04%	29.10%
Random Subsample	28.41%	35.21%	15.43%	26.35%
Filtered (Ours)	**47.89%**	**56.12%**	**38.95%**	**47.65%**
Net Gain vs. Full	*+16.47%*	*+17.28%*	*+21.91%*	*+18.55%*

To evaluate the cross-domain generalization and practical effectiveness of the VeriMM dataset, we conducted comprehensive benchmarking on three representative hardware design datasets: VerilogEval [12], RTLLM [14], and ChipGPT-V [2]. To ensure the integrity of our experimental results and prevent any potential data leakage, we implemented a rigorous filtering process that strictly excluded all samples belonging to these test sets from our training corpus.

The experimental results in Table 4 demonstrate that models fine-tuned on VeriMM consistently outperform base models across all benchmarks. The substantial performance gain on ChipGPT-V, which heavily relies on visual understanding, further validates the necessity of our multi-modal data in bridging the gap between natural language specifications and executable RTL code.

Table 4: Comparison of Pass@5 scores (%) before and after fine-tuning Qwen3-VL-8B on the VeriMM dataset across human-authored benchmarks.

Benchmark	Input Modality	Pre-tuned	Post-tuned (VeriMM)
Pure-Text Benchmarks			
VerilogEval [12]	Text	38.2	**64.6** (+ 26.4)
RTLLM [14]	Text	31.5	**50.9** (+ 19.4)
Multi-modal Benchmarks			
ChipGPT-V (Sch) [2]	Text + Img	24.2	**46.5** (+ 22.3)
ChipGPT-V (FSM) [2]	Text + Img	21.8	**43.1** (+ 21.3)

5 Conclusion

In a professional hardware design workflow, designers must interpret multimodal specifications—combining text and imagery—to implement Verilog code and verification environments. However, the scarcity of multimodal hardware datasets has hindered the ability of MLLMs to automate this process. We address this gap by proposing an automated image generation pipeline (producing timing diagrams, schematics, and FSMs) and an efficient testbench generation flow. Using these methods, we curated a large-scale dataset of 57k multimodal circuits. Our fine-tuning experiments demonstrate that this dataset significantly enhances the proficiency of MLLMs in hardware design tasks.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (Grant No. U25A20486 and No. 62306093).

References

[1] Sam Bush, Matthew DeLorenzo, Phat Tieu, and Jeyavijayan Rajendran. 2025. Free and Fair Hardware: A Pathway to Copyright Infringement-Free Verilog Generation using LLMs. arXiv:2505.06096 [cs.AI] https://arxiv.org/abs/2505.06096

[2] Kaiyan Chang, Zhirong Chen, Yunhao Zhou, Wenlong Zhu, Kun Wang, Haobo Xu, Cangyuan Li, Mengdi Wang, Shengwen Liang, Huawei Li, Yinhe Han, and Ying Wang. 2025. Natural language is not enough: Benchmarking multi-modal generative AI for Verilog generation. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design* (Newark Liberty International Airport Marriott, New York, NY, USA) *(ICCAD '24)*. Association for Computing Machinery, New York, NY, USA, Article 117, 9 pages. doi:10.1145/3676536.3676679

[3] Aliaksei Chapyzhenka. 2016. WaveDrom: Digital timing diagram engine. In *2016 Loughborough Antennas & Propagation Conference (LAPC)*. 1–5. doi:10.1109/LAPC.2016.7807530

[4] Fan Cui, Chenyang Yin, Kexing Zhou, Youwei Xiao, Guangyu Sun, Qiang Xu, Qipeng Guo, Yun Liang, Xingcheng Zhang, Demin Song, and Dahua Lin. 2025. *OriGen: Enhancing RTL Code Generation with Code-to-Code Augmentation and Self-Reflection*. Association for Computing Machinery, New York, NY, USA. https://doi.org/10.1145/3676536.3676830

[5] Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. 2024. Evaluating Large Language Models in Class-Level Code Generation. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) *(ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 81, 13 pages. doi:10.1145/3597503.3639219

[6] John Ellson, Emden R. Gansner, Eleftherios Koutsofios, Stephen C. North, and Gordon Woodhull. 2001. Graphviz — Open Source Graph Drawing Tools. In *Lecture Notes in Computer Science (Graph Drawing)*. Springer-Verlag, 483–484.

[7] Mingzhe Gao, Jieru Zhao, Zhe Lin, Wenchao Ding, Xiaofeng Hou, Yu Feng, Chao Li, and Minyi Guo. 2024. AutoVCoder: A Systematic Framework for Automated Verilog Code Generation using LLMs. In *2024 IEEE 42nd International Conference on Computer Design (ICCD)*. 162–169. doi:10.1109/ICCD63220.2024.00033

[8] Ananya Ghosh and K. Deepa. 2024. QueryMintAI: Multipurpose Multimodal Large Language Models for Personal Data. *IEEE Access* 12 (2024), 144631–144651. doi:10.1109/access.2024.3468996

[9] Google DeepMind. 2026. Gemini 3 Flash Preview. https://deepmind.google/technologies/gemini/. Accessed: 2026-01-27.

[10] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Kai Dang, et al. 2024. Qwen2. 5-Coder Technical Report. *arXiv preprint arXiv:2409.12186* (2024).

[11] Inkscape Project. 2026. *Inkscape Vector Graphics Editor*. Available at https://inkscape.org/.

[12] Mingjie Liu, Nathaniel Pinckney, Brucek Khailany, and Haoxing Ren. 2023. Invited Paper: VerilogEval: Evaluating Large Language Models for Verilog Code Generation. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. 1–8. doi:10.1109/ICCAD57390.2023.10323812

[13] Yi Liu, Changran Xu, Yunhao Zhou, Zeju Li, and Qiang Xu. 2025. DeepRTL: Bridging Verilog Understanding and Generation with a Unified Representation Model. arXiv:2502.15832 [cs.AR] https://arxiv.org/abs/2502.15832

[14] Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. 2024. RTLLM: An Open-Source Benchmark for Design RTL Generation with Large Language Model. In *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. 722–727. doi:10.1109/ASP-DAC58780.2024.10473904

[15] Bardia Nadimi, Ghali Omar Boutaib, and Hao Zheng. 2025. PyraNet: A Multi-Layered Hierarchical Dataset for Verilog. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. 1–7. doi:10.1109/DAC63849.2025.11133406

[16] OpenCores Community. 2026. OpenCores: The world's largest site for Open Source Hardware IP Cores. https://opencores.org. Accessed: 2026-01-27.

[17] Nathaniel Pinckney, Chenhui Deng, Chia-Tung Ho, Yun-Da Tsai, Mingjie Liu, Wenfei Zhou, Brucek Khailany, and Haoxing Ren. 2025. Comprehensive Verilog Design Problems: A Next-Generation Benchmark Dataset for Evaluating Large Language Models and Agents on RTL Design and Verification. arXiv:2506.14074 [cs.LG] https://arxiv.org/abs/2506.14074

[18] Ruidi Qiu, Grace Li Zhang, Rolf Drechsler, Tsungyi Ho, Ulf Schlichtmann, and Bing Li. 2025. ConfiBench: Automatic Testbench Generation with Confidence-Based Scenario Mask and Testbench Ensemble using LLMs for HDL Design. *ACM Trans. Des. Autom. Electron. Syst.* (Oct. 2025). doi:10.1145/3773087 Just Accepted.

[19] Ruidi Qiu, Grace Li Zhang, Rolf Drechsler, Ulf Schlichtmann, and Bing Li. 2024. AutoBench: Automatic Testbench Generation and Evaluation Using LLMs for HDL Design. In *Proceedings of the 2024 ACM/IEEE International Symposium on Machine Learning for CAD* (Salt Lake City, UT, USA) *(MLCAD '24)*. Association for Computing Machinery, New York, NY, USA, Article 18, 10 pages. doi:10.1145/3670474.3685956

[20] Ruidi Qiu, Grace Li Zhang, Rolf Drechsler, Ulf Schlichtmann, and Bing Li. 2025. CorrectBench: Automatic Testbench Generation with Functional Self-Correction using LLMs for HDL Design. In *2025 Design, Automation Test in Europe Conference (DATE)*. 1–7. doi:10.23919/DATE64628.2025.10992873

[21] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics. http://arxiv.org/abs/1908.10084

[22] Synopsys, Inc. 2026. Synopsys Verdi Debug Platform. https://www.synopsys.com/verification/debug/verdi.html. Accessed: 2026-01-27.

[23] Qwen Team. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] https://arxiv.org/abs/2505.09388

[24] V Team, Wenyi Hong, Wenmeng Yu, Xiaotao Gu, Guo Wang, Guobing Gan, Haomiao Tang, Jiale Cheng, Ji Qi, Junhui Ji, Lihang Pan, Shuaiqi Duan, Weihan Wang, Yan Wang, Yean Cheng, Zehai He, Zhe Su, Zhen Yang, Ziyang Pan, Aohan Zeng, Baoxu Wang, Bin Chen, Boyan Shi, Changyu Pang, Chenhui Zhang, Da Yin, Fan Yang, Guoqing Chen, Jiazheng Xu, Jiale Zhu, Jiali Chen, Jing Chen, Jinhao Chen, Jinghao Lin, Jinjiang Wang, Junjie Chen, Leqi Lei, Letian Gong, Leyi Pan, Mingdao Liu, Mingde Xu, Mingzhi Zhang, Qinkai Zheng, Sheng Yang, Shi Zhong, Shiyu Huang, Shuyuan Zhao, Siyan Xue, Shangqin Tu, Shengbiao Meng, Tianshu Zhang, Tianwei Luo, Tianxiang Hao, Tianyu Tong, Wenkai Li, Wei Jia, Xiao Liu, Xiaohan Zhang, Xin Lyu, Xinyue Fan, Xuancheng Huang, Yanling Wang, Yadong Xue, Yanfeng Wang, Yanzi Wang, Yifan An, Yifan Du, Yiming Shi, Yiheng Huang, Yilin Niu, Yuan Wang, Yuanchang Yue, Yuchen Li, Yutao Zhang, Yuting Wang, Yu Wang, Yuxuan Zhang, Zhao Xue, Zhenyu Hou, Zhengxiao Du, Zihan Wang, Peng Zhang, Debing Liu, Bin Xu, Juanzi Li, Minlie Huang, Yuxiao Dong, and Jie Tang. 2025. GLM-4.5V and GLM-4.1V-Thinking: Towards Versatile Multimodal Reasoning with Scalable Reinforcement Learning. arXiv:2507.01006 [cs.CV] https://arxiv.org/abs/2507.01006

[25] Shailja Thakur, Baleegh Ahmad, Hammond Pearce, Benjamin Tan, Brendan Dolan-Gavitt, Ramesh Karri, and Siddharth Garg. 2024. VeriGen: A Large Language Model for Verilog Code Generation. *ACM Trans. Des. Autom. Electron. Syst.* 29, 3, Article 46 (April 2024), 31 pages. doi:10.1145/3643681

[26] Toroid-io. 2026. vcd2wavedrom: Python script to transform a VCD file to WaveDrom format. https://github.com/Toroid-io/vcd2wavedrom. GitHub repository, accessed 2026-01-27.

[27] Nicholas Turley. 2026. netlistsvg: draws an SVG schematic from a JSON netlist. https://github.com/nturley/netlistsvg. GitHub repository, accessed 2026-01-27.

[28] Umama, Kamaluddeen Usman Danyaro, Maged Nasser, Abubakar Zakari, Shamsu Abdullahi, Atika Khanzada, Muhammad Muntasir Yakubu, and Sara Shoaib. 2025. LLM-Based Code Generation: A Systematic Literature Review With Technical and Demographic Insights. *IEEE Access* 13 (2025), 194915–194939. doi:10.1109/ACCESS.2025.3631952

[29] Ning Wang, Bingkun Yao, Jie Zhou, Yuchen Hu, Xi Wang, Zhe Jiang, and Nan Guan. 2025. Large Language Model for Verilog Generation with Code-Structure-Guided Reinforcement Learning. In *2025 IEEE International Conference on LLM-Aided Design (ICLAD)*. 164–170. doi:10.1109/ICLAD65226.2025.00025

[30] Weiyun Wang, Zhangwei Gao, Lixin Gu, Hengjun Pu, Long Cui, Xingguang Wei, Zhaoyang Liu, Linglin Jing, Shenglong Ye, Jie Shao, et al. 2025. InternVL3.5: Advancing Open-Source Multimodal Models in Versatility, Reasoning, and Efficiency. *arXiv preprint arXiv:2508.18265* (2025).

[31] Yiting Wang, Guoheng Sun, Wanghao Ye, Gang Qu, and Ang Li. 2025. VeriReason: Reinforcement Learning with Testbench Feedback for Reasoning-Enhanced Verilog Generation. arXiv:2505.11849 [cs.AI] https://arxiv.org/abs/2505.11849

[32] Stephen Williams. 2024. *Icarus Verilog User Guide*. Icarus Verilog Project, http://iverilog.icarus.com/. Version 12.0.

[33] Claire Wolf, Johann Glaser, and Friedrich Meisel. 2013. Yosys - A Free Verilog Synthesis Suite. In *Proceedings of Austrochip 2013*. Linz, Austria.

[34] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zhihao Fan. 2024. Qwen2 Technical Report. *arXiv preprint arXiv:2407.10671* (2024).

[35] Yiyao Yang, Fu Teng, Pengju Liu, Mengnan Qi, Chenyang Lv, Ji Li, Xuhong Zhang, and Zhezhi He. 2025. HaVen: Hallucination-Mitigated LLM for Verilog Code Generation Aligned with HDL Engineers. In *2025 Design, Automation Test in Europe Conference (DATE)*. 1–7. doi:10.23919/DATE64628.2025.10993072

[36] Yaoyu Zhu, Di Huang, Hanqi Lyu, Xiaoyun Zhang, Chongxiao Li, Wenxuan Shi, Yutong Wu, Jianan Mu, Jinghua Wang, Yang zhao, Pengwei Jin, Shuyao Cheng, shengwen Liang, Xishan Zhang, Rui Zhang, Zidong Du, Qi Guo, Xing Hu, and Yunji Chen. 2025. QiMeng-CodeV-R1: Reasoning-Enhanced Verilog Generation. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. https://openreview.net/forum?id=ly5DnRIgCZ